Language Features

OAVS Computer Teacher Competitive Exam — Revision Notes (Java)

“Language Features” here isn't a single Java concept — it's the umbrella name for a group of core Java syntax/semantics topics in Java. It covers the fundamental building blocks of how the Java language itself works, as opposed to specific APIs or libraries.

Contents

1. Scope Rules
2. Static Data, Static Methods, Static Blocks
3. All Modifiers of Class, Method, Data Members and Variable
4. Abstract Classes
5. Interfaces
6. Inner Classes
7. Wrapper Classes
8. Packages
9. Package Access
10. Importing Packages and Classes
11. User Define Packages

1 Scope Rules

1.1 Definition and Fundamental Concept

The **scope** of a variable (or identifier) is the region of the program within which it is declared and can be accessed directly by its name. In Java, scope is determined primarily by the **block structure** of the program - every pair of curly braces `{ }` defines a new block, and a new scope begins.

Scope decides **visibility** and **lifetime**: a variable is visible only inside the block in which it is declared (and any nested blocks inside it), and it is destroyed as soon as control leaves that block.

In simple terms: it gets deleted as soon as the program moves past that block.

1.2 Types of Scope in Java

Scope Type	Where Declared	Visible In	Lifetime
Class scope (Instance/Static variable)	Inside class, outside any method	Entire class (and outside via object/class name, subject to access modifier)	Instance: as long as object exists. Static: as long as class is loaded
Method scope (Local variable)	Inside a method body	Only within that method	Created on method call, destroyed on method return
Block scope	Inside { } such as if, for, while	Only within that block	Created on block entry, destroyed on block exit
Parameter scope	In method/constructor parameter list	Entire body of that method/constructor	Duration of the method call
Loop scope	In for(...) initialization	Only within the loop body	Re-created each loop (for loop counter)

1.3 Working Mechanism / Rules

- A variable declared inside a block is **local** to that block; it cannot be accessed outside it.
- Inner blocks can access variables of the enclosing (outer) block, but not vice-versa.
- A local variable **must be initialized** before use - Java does not give local variables a default value (unlike instance/static variables).
- You **cannot redeclare** a variable with the same name in a nested block if it is already declared in the enclosing block (compile-time error - duplicate variable).
- Instance and static variables get default values automatically (0, 0.0, false, null) and follow class scope.
- Parameters have scope limited to the method/constructor in which they are declared.

1.4 Java Example

```java
class ScopeDemo {
    int x = 10; // instance scope - whole class
    static int y = 20; // static/class scope

    void display() {
        int a = 5; // local/method scope
        if (a > 0) {
            int b = 15; // block scope (only inside if)
            System.out.println(a + b + x + y);
        }
        // b is NOT accessible here - out of scope
        System.out.println(a); // OK, a is method scope
    }
}
public class Main {
    public static void main(String[] args) {
        new ScopeDemo().display();
    }
}
```

Output:

```
50
```

1.5 Exam-Oriented Facts

EXAM POINT

Important: Java has NO true global variables like C/C++. The nearest equivalent is a **public static** variable, which is accessible via ClassName.variable from anywhere the class is visible.

COMMON CONFUSION

Common Mistake: Trying to use a variable outside its block (e.g., a for-loop counter used after the loop ends) causes a compile-time error: *cannot find symbol*.

SUMMARY

Scope = region of code where an identifier is valid. Java scopes: class, method, block, parameter. Inner blocks see outer variables; outer blocks cannot see inner ones. Local variables need explicit initialization; instance/static variables get defaults.

2 Static Data, Static Methods, Static Blocks

2.1 The 'static' Keyword - Core Idea

static means the member belongs to the **class itself**, not to any individual object. Only **one copy** of a static member exists, shared by all instances of the class. Static members are loaded into memory when the class is loaded by the JVM - before any object is created.

2.2 Static Data (Static Variables / Class Variables)

- Declared using the `static` keyword inside a class, outside any method.
- Shared among all objects - if one object changes it, the change reflects for all objects.
- Memory allocated **once**, at class-loading time, stored in the method area (class area).
- Accessed using `ClassName.variableName` (preferred) or `objectName.variableName`.
- Gets a default value automatically if not explicitly initialized.

```
class Counter {
    static int count = 0; // static data member
    Counter() { count++; }
}
public class Main {
    public static void main(String[] args) {
        new Counter(); new Counter(); new Counter();
        System.out.println(Counter.count);
    }
}
```

Output:
3

2.3 Static Methods

- Declared with `static`; belong to the class, can be called without creating an object: `ClassName.methodName()`.
- Can access **only static data** and can call **only other static methods** of the same class directly.
- **Cannot use** `this` or `super` keywords (no object context).
- Cannot directly access instance (non-static) variables or instance methods.
- Commonly used for utility/helper methods, e.g., `Math.sqrt()`, `main()` itself is static.
- Static methods can be **overloaded** but their overriding is different - redeclaring a static method in a subclass causes **method hiding**, not runtime polymorphism.

```
class MathUtil {
    static int square(int n) { return n * n; }
}
public class Main {
    public static void main(String[] args) {
        System.out.println(MathUtil.square(6));
    }
}
```

Output:
36

2.4 Static Blocks (Static Initialization Blocks)

- A block of code prefixed with `static` keyword, used to initialize static data members with complex logic that a simple assignment cannot handle.
- Executes **only once**, when the class is **first loaded** into memory by the JVM.

- Executes **before the main() method** and before any constructor or object creation.
- A class can have **multiple static blocks** - they execute in the **order they appear** in the source code.

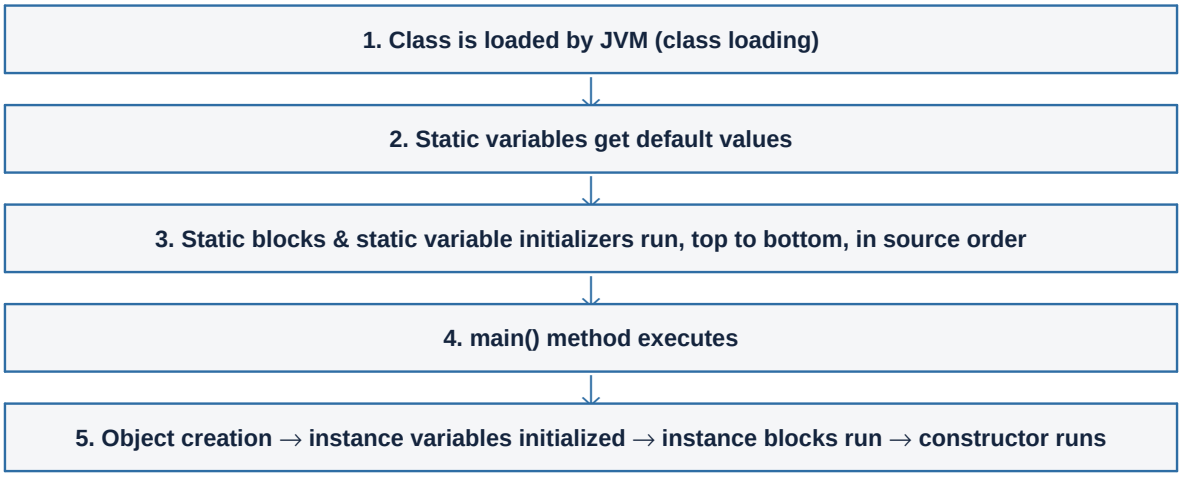
```
class Demo {
    static int x;
    static {
        x = 100;
        System.out.println("Static block executed");
    }
    Demo() { System.out.println("Constructor executed"); }
}
public class Main {
    public static void main(String[] args) {
        System.out.println("main started");
        new Demo();
    }
}
```

Output:

```
Static block executed
main started
Constructor executed
```

Execution order note: Although *main started* appears after the static block in output, the static block runs at class-loading time which happens before main() begins execution for that class - hence it always prints first.

2.5 Execution Order Diagram



2.6 Comparison Table

Feature	Static Member	Instance Member
Belongs to	Class	Object
Memory allocated	Once, at class loading	Every time an object is created
Accessed by	ClassName.member	objectName.member
Access to non-static members	Not allowed directly	Allowed
this/super keyword	Cannot be used	Can be used

2.7 Exam-Oriented Facts

EXAM POINT

Important: The `main()` method is **public static void** precisely because the JVM must call it without creating an object of the class first.

COMMON CONFUSION

Common Mistake: Writing `this.x` inside a static method - this is a **compile-time error** because static context has no 'this' reference.

COMMON CONFUSION

Common Mistake: Confusing static block with instance initializer block (instance block - without 'static' keyword - runs every time an object is created, right before the constructor).

SUMMARY

static = belongs to class, not object; one copy shared by all. Static data: class-level variable. Static method: callable without object, accesses only static members. Static block: runs once at class loading, before main(), used for static initialization; multiple blocks run in source order.

3 All Modifiers of Class, Method, Data Members and Variable

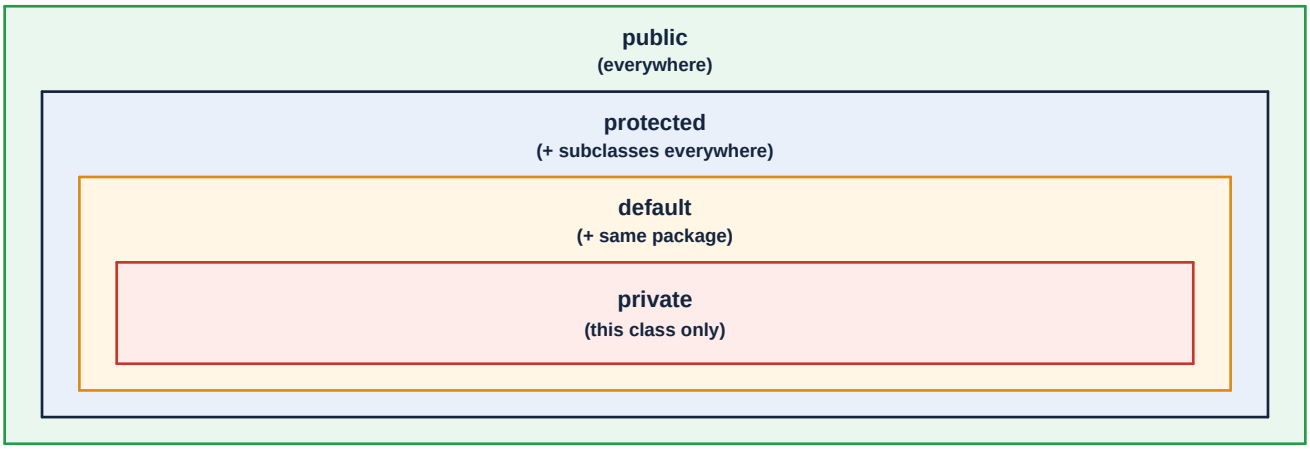
3.1 Definition and Fundamental Concept

Modifiers are keywords added to declarations of classes, methods, data members (fields) and variables to change their meaning - controlling **access** (who can use them) and **behaviour** (how they work). Java modifiers are divided into two broad categories:

- **Access Modifiers** - control visibility: `default (no modifier), public, private, protected`.
- **Non-Access Modifiers** - control behaviour: `static, final, abstract, synchronized, transient, volatile, native, strictfp`.

3.2 Access Modifiers - Visibility Diagram

The simple rule: **private** is the most hidden, **public** is the most open. Each outer ring can see everything the ring inside it can see, plus more.



Modifier	Same Class	Same Package	Subclass (Diff. Package)	Diff. Package (Non-subclass)
public	Yes	Yes	Yes	Yes
protected	Yes	Yes	Yes	No
default (no modifier)	Yes	Yes	No	No
private	Yes	No	No	No

How to read this: the rows go from **most open** (public) at the top to **most restricted** (private) at the bottom. Each column is a place code might try to access the member from; "Yes" means that location can see it, "No" means it cannot. Notice each row only ever loses access moving down - it never gains any back, since public → protected → default → private is a strictly narrowing sequence.

3.3 Access Modifiers Applicable to Class / Method / Data Member / Variable

Modifier	Top-level Class	Method	Data Member (Field)	Local Variable
public	Allowed	Allowed	Allowed	Not allowed
protected	Not allowed	Allowed	Allowed	Not allowed
default	Allowed	Allowed	Allowed	Not allowed (implicit)
private	Not allowed	Allowed	Allowed	Not allowed

EXAM POINT

Important: A **top-level class** can only be `public` or `default` - never private or protected. Only a **nested (inner) class** can be private or protected.

EXAM POINT

Important: Local variables cannot have any access modifier - their scope is already restricted to the block they are declared in.

3.4 Non-Access Modifiers - Applicability Table

These don't control who can see something - they control *how it behaves*. A quick way to group them: **static, final, abstract** shape how the class/method/field fits into the type system, while **synchronized, transient, volatile** each solve one specific runtime problem (thread safety, saving objects, and memory visibility).

Modifier	Class	Method	Data Member	Meaning
static	Yes (nested only)	Yes	Yes	Belongs to the class itself, not to any one object - every object shares the same single copy. (e.g. Math.PI - you never make a Math object to use it.)
final	Yes	Yes	Yes	Locks it - no further change allowed. Class: can't be subclassed. Method: can't be overridden. Variable: value can never change once assigned (a constant).
abstract	Yes	Yes	No	Deliberately incomplete - the details are left for later. Class: can't be built into an object directly. Method: has no body; a subclass must supply one. (Doesn't apply to fields - a variable can't be "incomplete", it either has a value or it doesn't.)
synchronized	No	Yes	No	A thread-safety lock: only one thread can run this method at a time, so two threads can't corrupt shared data by running it simultaneously.
transient	No	No	Yes	Tells Java to skip this field when the object is serialized (saved/sent as a byte stream) - its value is simply left out of the saved copy.
volatile	No	No	Yes	Stops threads from keeping a private cached copy of this field's value - every read/write always goes straight to main memory, so all threads see the latest value.
native	No	Yes	No	The method's real implementation lives outside Java, usually written in a language like C or C++.
strictfp	Yes	Yes	No	Forces decimal (floating-point) calculations to give the exact same result on every machine/JVM.

3.5 Important Modifier Combination Rules

- `abstract` and `final` can **never** be used together on a class or method - abstract demands implementation/extension, final forbids it (direct contradiction).
- `abstract` and `static` cannot be used together on a method - static methods are resolved at compile time, abstract methods need runtime implementation.
- `abstract` and `private` cannot be used together on a method - a private method is not visible to subclasses, so it can never be implemented/overridden.
- A `final` variable that is a primitive keeps its value constant forever; a `final` reference variable's *reference* is constant, but the object's internal state can still change.
- `static final` together (e.g., `public static final int MAX=100;`) is the standard way to declare a **constant** in Java.

3.6 Java Example

```java
class Account {
    private double balance; // private data member
    public static final double RATE = 4.5; // constant
    protected void showRate() {
        System.out.println("Rate: " + RATE);
    }
    public final void freeze() { // cannot be overridden
        System.out.println("Account frozen");
    }
}
public class Main {
    public static void main(String[] args) {
        Account a = new Account();
        a.showRate();
        a.freeze();
    }
}
```

Output:

```
Rate: 4.5
Account frozen
```

SUMMARY

Access modifiers (public, protected, default, private) control visibility across class/package/subclass boundaries. Non-access modifiers (static, final, abstract, synchronized, transient, volatile, native, strictfp) control behaviour. Local variables take no access modifier. Top-level classes: only public or default. Remember illegal combos: abstract+final, abstract+static, abstract+private.

4 Abstract Classes

4.1 Definition and Fundamental Concept

An **abstract class** is a class declared with the `abstract` keyword that **cannot be instantiated** directly (no objects can be created from it). It is designed to be **extended (subclassed)**, providing a common base with some implemented behaviour and some behaviour left to be defined by subclasses.

4.2 Purpose and Key Characteristics

- Achieves **partial abstraction** - can contain both abstract (no body) and concrete (with body) methods.
- Provides a common template/blueprint while forcing subclasses to implement specific behaviour.
- Can have **constructors** (called implicitly via subclass constructor using super()), instance variables, static members, and normal methods - unlike an interface.
- Can have any access modifier on its methods (public, protected, default) - not restricted like interfaces.
- If a class contains even **one abstract method**, the class itself **must** be declared abstract.
- A subclass of an abstract class must implement **all** abstract methods, or it too must be declared abstract.

4.3 Syntax and Declaration

```
abstract class Shape {
    abstract void draw(); // abstract method - no body
    void info() { // concrete method - has body
        System.out.println("This is a shape.");
    }
}
```

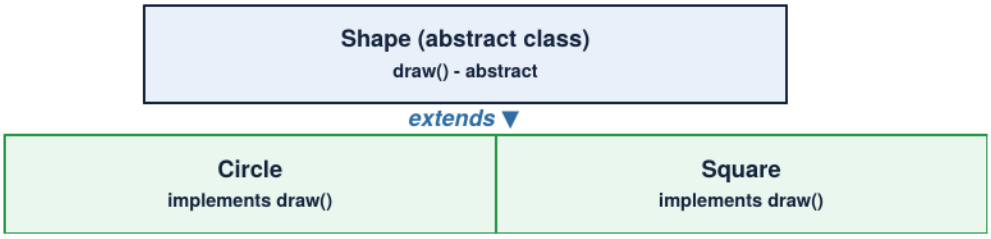
4.4 Java Example

```
abstract class Shape {
    abstract void draw();
}
class Circle extends Shape {
    void draw() { System.out.println("Drawing Circle"); }
}
class Square extends Shape {
    void draw() { System.out.println("Drawing Square"); }
}
public class Main {
    public static void main(String[] args) {
        Shape s1 = new Circle(); // upcasting
        Shape s2 = new Square();
        s1.draw();
        s2.draw();
    }
}
```

Output:

```
Drawing Circle
Drawing Square
```

4.5 Abstract Class Relationship Diagram



Element	Abstract Class (Shape)	Concrete Subclass (Circle / Square)
Instantiable?	No - cannot do 'new Shape()'	Yes - 'new Circle()' allowed
Abstract method draw()	Declared only (no body)	Must provide full implementation
Reference use	Shape s1 = new Circle(); (runtime polymorphism)	-

4.6 Exam-Oriented Facts

EXAM POINT

Important: An abstract class **can have a constructor**, even though it cannot be instantiated directly - the constructor runs when a subclass object is created (via implicit super()).

EXAM POINT

Important: An abstract class need **not** contain any abstract method - it can be declared abstract purely to prevent instantiation.

COMMON CONFUSION

Common Mistake: Writing `new Shape()` for an abstract class Shape - this is a **compile-time error**.

COMMON CONFUSION

Common Mistake: Forgetting to implement even one abstract method in a concrete subclass - the subclass will then also be forced to be declared abstract, or the code fails to compile.

SUMMARY

Abstract class = cannot be instantiated, declared using 'abstract' keyword, can mix abstract and concrete methods, can have constructors/fields, supports partial abstraction, subclass must implement all abstract methods.

5 Interfaces

5.1 Definition and Fundamental Concept

An **interface** is a completely abstract type in Java, declared using the `interface` keyword, that specifies a set of methods a class must implement. It represents a **contract** - '*what* to do', leaving '*how*' entirely to the implementing class. Interfaces achieve **full/100% abstraction** (for its traditional members) and support Java's form of **multiple inheritance of type**.

5.2 Key Characteristics

- All methods are **implicitly public and abstract** (unless declared default, static, or private - modern Java features).
- All fields/variables are **implicitly public, static and final** - i.e., constants; must be initialized at declaration.
- An interface **cannot have a constructor** and **cannot be instantiated**.
- A class uses `implements` keyword to use an interface; an interface uses `extends` keyword to inherit another interface (and an interface can extend **multiple** interfaces).
- A class can `implement multiple interfaces` - this is how Java achieves multiple inheritance safely (no diamond problem, since interfaces traditionally had no method bodies).
- Since Java 8: interfaces may also have `default` methods (with body) and `static` methods (with body). Since Java 9: `private` methods are allowed too.

5.3 Syntax

```
interface Printable {
    int MAX = 100; // public static final (implicit)
    void print(); // public abstract (implicit)
}
```

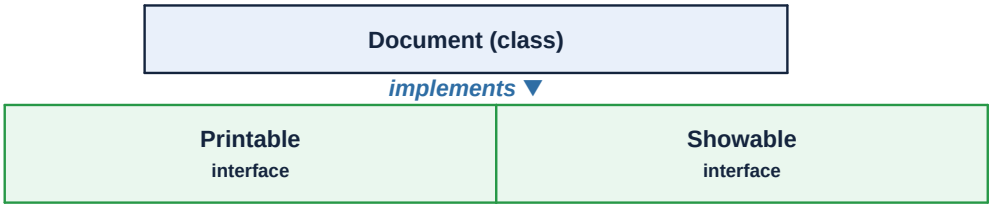
5.4 Java Example - Implementation

```
interface Printable {
    void print();
}
interface Showable {
    void show();
}
class Document implements Printable, Showable {
    public void print() { System.out.println("Printing document..."); }
    public void show() { System.out.println("Showing document..."); }
}
public class Main {
    public static void main(String[] args) {
        Document d = new Document();
        d.print();
        d.show();
    }
}
```

Output:

```
Printing document...
Showing document...
```

5.5 Interface Implementation Diagram



Relationship	Keyword Used	Multiplicity
Class implementing interface(s)	class X implements A, B	A class can implement multiple interfaces
Interface extending interface(s)	interface A extends B, C	An interface can extend multiple interfaces
Class extending class	class X extends Y	A class can extend only ONE class (single inheritance)

5.6 Abstract Class vs Interface (Exam Favourite)

Point	Abstract Class	Interface
Keyword	abstract class	interface
Methods	Abstract + concrete methods allowed	Traditionally only abstract; now default/static/private too
Variables	Any type (instance, static, final...)	Only public static final (constants)
Constructor	Allowed	Not allowed
Inheritance	extends (only one class)	implements (multiple interfaces allowed)
Access modifiers on methods	Any (public/protected/default)	public only (implicitly)
When to use	Related classes sharing common code/state	Unrelated classes sharing common behaviour/contract

5.7 Exam-Oriented Facts

EXAM POINT

Important: All interface variables are constants by default - `public static final` - so they must be assigned a value at declaration and cannot be reassigned.

COMMON CONFUSION

Common Mistake: Declaring an interface method as `private` or adding a method body without 'default'/'static' keyword (pre-Java 8 style) causes a compile-time error.

SUMMARY

Interface = full abstraction & contract; methods implicitly public abstract (default/static allowed since Java 8); fields implicitly public static final; no constructor; a class can implement multiple interfaces (multiple inheritance of type).

6 Inner Classes

6.1 Definition and Fundamental Concept

An **inner class** (nested class) is a class defined **inside another class**. It logically groups classes that are used in only one place, increases encapsulation, and can directly access the members (including private members) of the outer (enclosing) class.

6.2 Types / Forms of Inner Classes

Type	Declared	Needs Outer Object?	Key Point
Member (Non-static) Inner Class	Inside class, outside any method, without 'static'	Yes (to create an object)	Can access all outer members directly, including private
Static Nested Class	Inside class, with 'static' keyword	No (to create an object)	Behaves like a top-level class placed inside another for organization; can access only static members of outer class
Local Inner Class	Inside a method body	Yes, if used (to create an object)	Scope limited to the method/block; cannot use access modifiers
Anonymous Inner Class	No class name; declared and instantiated together, usually to implement an interface/abstract class on the fly	Yes/No (to create an object)	Used for one-time-use implementations, e.g., event handlers

Note: "Needs Outer Object?" refers specifically to **creating (instantiating)** an object of the inner class with `new` - not to declaring the class itself, which never requires an outer object.

6.3 Syntax and Examples

(a) Member (Non-static) Inner Class

```
class Outer {
    private int data = 10;
    class Inner { // non-static inner class
        void show() {
            System.out.println("Data = " + data); // accesses outer private field
        }
    }
}
public class Main {
    public static void main(String[] args) {
        Outer o = new Outer();
        Outer.Inner in = o.new Inner(); // note the syntax
        in.show();
    }
}
```

Output:

```
Data = 10
```

(b) Static Nested Class

```java
class Outer {
    static int x = 50;
    static class Nested {
        void show() { System.out.println("x = " + x); }
    }
}
public class Main {
    public static void main(String[] args) {
        Outer.Nested n = new Outer.Nested(); // no outer object needed
        n.show();
    }
}
```

Output:

```
x = 50
```

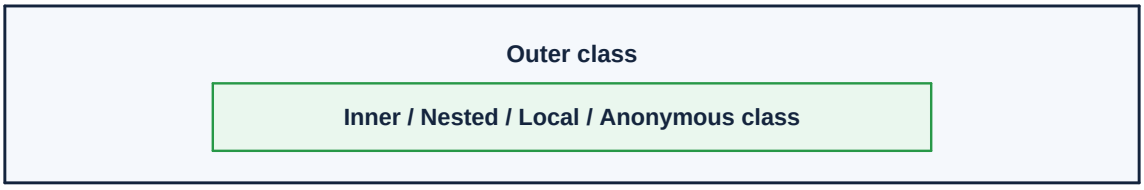
(c) Anonymous Inner Class

```java
interface Greet { void hello(); }
public class Main {
    public static void main(String[] args) {
        Greet g = new Greet() { // anonymous class implementing interface
            public void hello() { System.out.println("Hello!"); }
        };
        g.hello();
    }
}
```

Output:

```
Hello!
```

6.4 Inner Class Structure Diagram



Level	Class
Outermost	Outer (top-level class)
Nested, inside Outer	Inner / Nested / Local / Anonymous class
Compiled file names	Outer.class, Outer$Inner.class (compiler generates separate .class files using $ symbol)

6.5 Exam-Oriented Facts

EXAM POINT

Important: Object creation syntax for a non-static inner class is `Outer.Inner obj = outerObj.new Inner();` - an outer class object **must exist first**.

EXAM POINT

Important: A **static nested class** does NOT need an outer class object: `Outer.Nested obj = new Outer.Nested();`

COMMON CONFUSION

Common Mistake: Trying to access non-static outer members from a static nested class - not allowed, since static nested class has no implicit reference to an outer object.

SUMMARY

Inner class = class defined inside another class. Four forms: member (non-static), static nested, local, anonymous. Non-static inner class needs outer object and can access all outer members (even private); static nested class behaves independently and accesses only static outer members.

7 Wrapper Classes

7.1 Definition and Fundamental Concept

A **wrapper class** is a class that wraps (encloses) a primitive data type into an object. Java provides one wrapper class for each of the eight primitive types, all present in the `java.lang` package.

7.2 Purpose

- Java Collections (ArrayList, HashMap, etc.) work only with **objects**, not primitives - wrapper classes let primitives be stored in collections.
- Enable primitives to be treated as objects for use with generics, and to use utility methods (e.g., parsing strings to numbers).
- Provide useful constants (e.g., `Integer.MAX_VALUE`) and utility methods (e.g., `Integer.parseInt()`).
- Support **Autoboxing** (automatic primitive-to-wrapper conversion) and **Unboxing** (automatic wrapper-to-primitive conversion), introduced in Java 5.

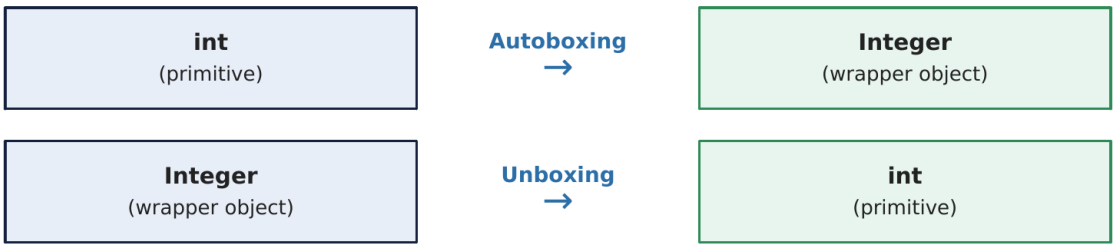
"Generics" in Java let you write classes, interfaces, and methods that work with a type you specify later, instead of hard-coding a specific type.

Think of ArrayList before generics existed: it could hold anything, so you'd write `ArrayList list = new ArrayList();`, add a `String`, and the compiler wouldn't stop you from also adding an `Integer` by mistake — you'd only find out at runtime when a cast failed.

7.3 Primitive Type - Wrapper Class Mapping

Primitive Type	Wrapper Class
byte	Byte
short	Short
int	Integer
long	Long
float	Float
double	Double
char	Character
boolean	Boolean

7.4 Wrapper Class Relationship Diagram



7.5 Primitive to Wrapper (Boxing)

```
int a = 25;
Integer obj1 = Integer.valueOf(a); // explicit boxing
Integer obj2 = a; // autoboxing (implicit, Java 5+)
System.out.println(obj1 + " " + obj2);
```

Output:

```
25 25
```

7.6 Wrapper to Primitive (Unboxing)

```
Integer obj = 50;
int b = obj.intValue(); // explicit unboxing
int c = obj; // auto-unboxing (implicit)
System.out.println(b + " " + c);
```

Output:

```
50 50
```

7.7 Useful Wrapper Methods (Exam Favourites)

Method	Purpose	Example
parseInt(String)	Converts String to int (primitive)	int n = Integer.parseInt("123");
valueOf(...)	Returns a wrapper object for the given value/String	Integer i = Integer.valueOf("45");
toString()	Converts wrapper/primitive value to String	String s = Integer.toString(10);
intValue(), doubleValue()...	Returns primitive value from wrapper object	int x = obj.intValue();
MAX_VALUE / MIN_VALUE	Constants giving range of the type	Integer.MAX_VALUE = 2147483647

7.8 Exam-Oriented Facts

EXAM POINT

Important: Wrapper classes are **immutable** - once a wrapper object is created, its value cannot be changed; a new object is created for a new value.

COMMON CONFUSION

Common Mistake: Using `==` to compare wrapper objects can give unexpected results for values outside the cached range (-128 to 127 for Integer) - always use `.equals()` to compare wrapper object values.

SUMMARY

Wrapper classes convert primitives into objects (one per primitive type, in java.lang). Autoboxing = primitive to object automatically; Unboxing = object to primitive automatically. Needed for Collections, generics, and utility methods like parseInt().

8 Packages

8.1 Definition and Fundamental Concept

A **package** in Java is a **namespace/folder** mechanism that groups related classes, interfaces, and sub-packages together. It is Java's way of organizing classes into a directory structure and avoiding naming conflicts between classes.

8.2 Purpose

- **Organizes** classes/interfaces into manageable, logical groups.
- **Avoids naming collisions** - two classes with the same name can exist in different packages.
- Provides **access protection** via the default (package-private) access level.
- Makes classes easier to **locate and reuse**.

8.3 Types of Packages

Type	Description	Example
Built-in (Predefined) Package	Supplied by Java itself (part of JDK)	java.lang, java.util, java.io, java.awt
User-defined Package	Created by the programmer for their own classes	com.school.students

8.4 Package Declaration Syntax

```
package packageName; // must be the VERY FIRST statement in the .java file

public class MyClass {
    // class body
}
```

IMPORTANT

The package statement must be the **very first statement** in the .java file - nothing except comments may come before it, not even an import or class declaration.

EXAM POINT

Important: Only **one** package statement is allowed per file, and it must appear before any import statements or class declarations (comments may precede it).

One package statement, multiple imports (example)

```
package mypack;              // this file's own package (only one)

import java.util.ArrayList;  // from java.util
import java.io.File;         // from java.io
import com.oavs.exam.Result; // from com.oavs.exam

public class Greet {
    ArrayList<String> list = new ArrayList<>();
    File f = new File("data.txt");
    Result r = new Result();
}
```

IMPORTANT

The `package` statement is **optional**. If omitted, the class belongs to the **default (unnamed) package** - imports still work normally, since imports bring in classes from other packages regardless of your own file's package. However, a class in the default package can never be imported by a class in a named package.

8.5 Package Hierarchy / Directory Structure Diagram

A package name maps directly to a **directory (folder) structure** on disk. `package com.school.students;` requires the .class file to physically reside in the folder path: `com/school/students/`

```
com/
  |-- school/
      |-- students/
          |-- Result.class (package com.school.students;)
```

Package Statement	Required Folder Path	Fully Qualified Class Name
package shapes;	shapes/	shapes.Circle
package com.oavs.exam;	com/oavs/exam/	com.oavs.exam.Result

8.6 Exam-Oriented Facts

- If no package statement is used, the class belongs to the **default (unnamed) package**.
- Package names are conventionally written in **lowercase** to avoid conflicts with class names.
- `java.lang` package is **imported automatically** into every Java program - no explicit import needed (e.g., String, System, Math, wrapper classes).

SUMMARY

Package = namespace grouping related classes/interfaces; maps to a folder structure on disk. Types: built-in (java.util, java.io...) and user-defined. 'package' statement must be the first line in the file.

9 Package Access

9.1 Definition and Fundamental Concept

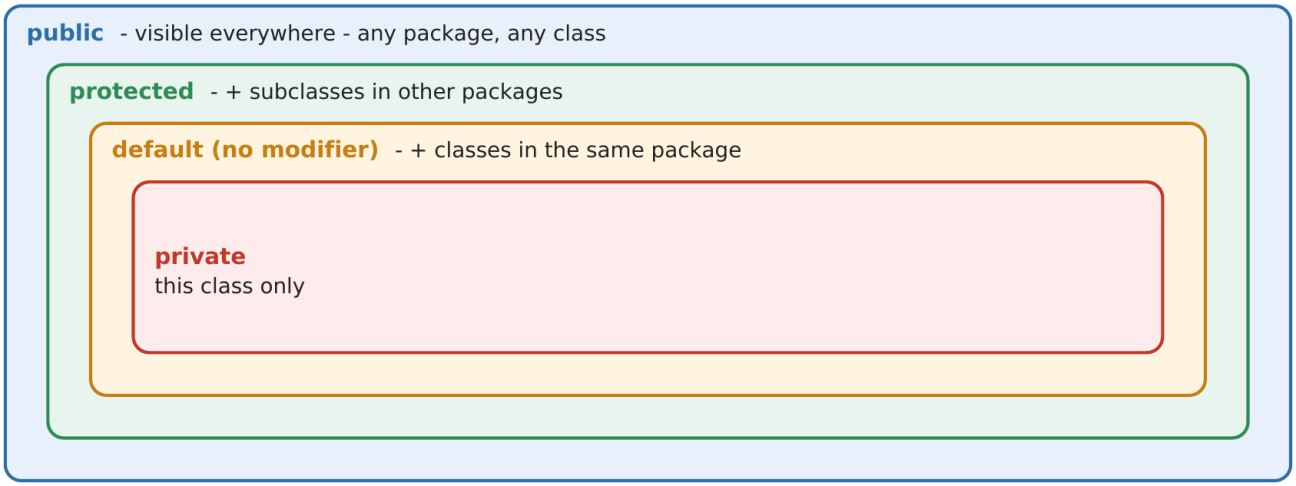
Package access refers to the rules governing which classes/members declared in one package can be accessed from classes in another package. It directly depends on the **access modifier** used and, for classes, whether they are declared public.

9.2 Access Rules Across Packages

Modifier on Member	Same Class	Same Package (Different Class)	Subclass in Different Package	Different Package (Non-subclass)
public	Accessible	Accessible	Accessible	Accessible
protected	Accessible	Accessible	Accessible (via inheritance)	Not accessible
default (no modifier)	Accessible	Accessible	Not accessible	Not accessible
private	Accessible	Not accessible	Not accessible	Not accessible

9.3 Package Access Diagram (Conceptual)

- **public class/member**: visible everywhere - any package, any class.
- **protected member**: visible within the same package PLUS to subclasses in other packages (via inheritance only, not via a plain object reference).
- **default class/member**: visible only to classes within the *same* package - invisible outside, even to subclasses in other packages.
- **private member**: visible only inside its own class - not even to other classes in the same package.



Each outer ring can see everything the ring inside it can see, plus more. Order: public → protected → default → private (most open to most restricted).

9.4 Java Example

```
// File: pkgA/Base.java
package pkgA;
public class Base {
    protected void greet() { System.out.println("Hello from Base"); }
}

// File: pkgB/Derived.java
package pkgB;
import pkgA.Base;
public class Derived extends Base {
    public static void main(String[] args) {
        Derived d = new Derived();
        d.greet(); // accessible because Derived IS-A Base (protected via inheritance)
    }
}
```

Output:

```
Hello from Base
```

9.5 Exam-Oriented Facts

COMMON CONFUSION

Common Mistake: A class must be declared `public` to be used outside its own package. A **default-access (non-public) class** cannot even be imported and used from another package.

EXAM POINT

Important: A `.java` source file can contain only **one** public top-level class, and its name must match the file name.

SUMMARY

Package access = visibility rules across package boundaries, governed by access modifiers. Only public members/classes are accessible outside their package; protected adds subclass access across packages; default and private never cross package boundaries (default not even to non-package subclasses).

10 Importing Packages and Classes

10.1 Definition and Purpose

The `import` statement allows a Java program to use classes and interfaces defined in another package **without typing their fully qualified name** every time. It must appear after the package statement (if any) and before any class declarations.

10.2 Syntax

Form	Syntax	Effect
Import a single class	import packageName.ClassName;	Only the named class becomes accessible by simple name
Import all classes of a package	import packageName.*;	All public classes/interfaces of that package become accessible (does NOT import sub-packages)
Fully Qualified Name (no import)	packageName.ClassName obj = ...	Used directly without any import statement
Static import	import static packageName.ClassName.member;	Allows static members to be used without class-name prefix

10.3 Java Example

```
import java.util.ArrayList; // import single class
import java.util.*; // import all classes of java.util

public class Main {
    public static void main(String[] args) {
        ArrayList<String> list = new ArrayList<>();
        list.add("OAVS");
        list.add("Exam");
        System.out.println(list);
    }
}
```

Output:

```
[OAVS, Exam]
```

10.4 Static Import Example

```
import static java.lang.Math.sqrt;
import static java.lang.Math.PI;

public class Main {
    public static void main(String[] args) {
        System.out.println(sqrt(16)); // no need to write Math.sqrt
        System.out.println(PI);
    }
}
```

Output:

```
4.0
3.141592653589793
```

10.5 Package Importing Flow Diagram

1. Compiler reads the 'package' statement (if present) - identifies the current package

↓

2. Compiler reads all 'import' statements - builds a lookup table of accessible class names

↓

3. While compiling the class body, resolves each simple class name using the lookup table (imports), java.lang (auto-imported), or the current package

↓

4. If the class name cannot be resolved → 'cannot find symbol' compile error

10.6 Exam-Oriented Facts

- `import java.util.*;` does **not** import sub-packages, e.g., `java.util.concurrent` must be imported separately.
- Importing with `*` does **not** reduce runtime performance - it only affects compile-time name resolution; the compiled .class file is identical either way.
- If two imported packages contain classes with the **same name** (e.g., java.util.Date and java.sql.Date), you must use the **fully qualified name** to disambiguate.
- `java.lang.*` is imported automatically by the compiler in every Java file.

COMMON CONFUSION

Common Mistake: Placing an import statement *after* a class declaration causes a compile-time error - imports must come before any type declarations.

SUMMARY

import brings classes/interfaces of other packages into scope by simple name. Forms: single-class import, wildcard (*) import, static import. java.lang is auto-imported; wildcard import does not include sub-packages.

11 User Define Packages

11.1 Definition and Fundamental Concept

A **user-defined package** is a package created by the programmer to organize their own classes and interfaces, as opposed to Java's built-in packages (java.util, java.io, etc.).

11.2 Steps to Create and Use a User-Defined Package

Step	Action	Command / Code
1	Declare the package as the first statement in the .java file	package mypack;
2	Write the class(es) that belong to the package (usually declared public to be usable outside)	public class Greet { ... }
3	Compile the file with the -d option to place the .class file in the correct folder structure	javac -d . Greet.java
4	This creates a folder named 'mypack' containing Greet.class	mypack/Greet.class
5	Use the package in another program via import	import mypack.Greet;
6	Compile and run the program that uses the package (ensure classpath includes the package root)	javac Main.java then java Main

11.3 Java Example - Creating a User-Defined Package

Step 1: Create the package member class

```
// File: Greet.java
package mypack;
public class Greet {
    public void wish() {
        System.out.println("Welcome to OAVS!");
    }
}
```

Compile using: `javac -d . Greet.java` → creates folder `mypack/` with `Greet.class` inside it.

Step 2: Use the package in another class

```
// File: Main.java
import mypack.Greet;
public class Main {
    public static void main(String[] args) {
        Greet g = new Greet();
        g.wish();
    }
}
```

Output:

```
Welcome to OAVS!
```

11.4 Package Organization Rules

- Package names should be **all lowercase** to avoid confusion with class names (convention).
- For company/organization-level uniqueness, package names commonly follow the reversed domain-name convention, e.g., `com.oavs.exam`.
- A **sub-package** is created simply by adding another level, e.g., `package mypack.utils;` → folder path `mypack/utils/`.

- All classes intended to be used outside the package **must be declared public**; classes without 'public' are only usable within the same package (default access).

11.5 Exam-Oriented Facts

EXAM POINT

Important: The `-d` option in `javac -d . Greet.java` tells the compiler **where to place** the generated package directory structure ('.' means current directory).

COMMON CONFUSION

Common Mistake: Forgetting to make the package class or its members `public` - the importing class in another package will then get a compile-time 'not public in ... cannot be accessed from outside package' error.

COMMON CONFUSION

Common Mistake: Placing the .java file in the wrong folder relative to its package declaration causes a runtime *NoClassDefFoundError* or compile-time errors when the package is imported.

SUMMARY

User-defined package: created via 'package name;' declaration, compiled with 'javac -d .', producing a matching folder structure; classes must be public to be used outside the package; imported elsewhere using 'import packageName.ClassName;'.

QUICK REVISION - Language Features

A. Key Definitions at a Glance

Term	One-Line Definition
Scope	Region of code where a declared identifier is valid and accessible
Static member	Belongs to the class (one shared copy), not to individual objects
Static block	Runs once, at class loading, before main() - used for static initialization
Access modifier	Controls visibility: public, protected, default, private
Non-access modifier	Controls behaviour: static, final, abstract, synchronized, transient, volatile, native, strictfp
Abstract class	Cannot be instantiated; may mix abstract & concrete methods; supports partial abstraction
Interface	Full abstraction contract; methods public abstract by default; fields public static final
Inner class	Class defined inside another class - member, static nested, local, or anonymous
Wrapper class	Object representation of a primitive type (Integer for int, etc.)
Autoboxing / Unboxing	Automatic primitive-to-object / object-to-primitive conversion
Package	Namespace grouping related classes; maps to a directory structure
Package access	Visibility rules for members/classes across package boundaries
import	Brings classes of another package into scope by simple name
User-defined package	Programmer-created package compiled with 'javac -d .'

B. Core Syntax Cheat-Sheet

```
package mypack; // package declaration (must be first line)
import mypack.Greet; // import a class
import mypack.*; // import all classes of a package
import static java.lang.Math.sqrt; // static import

abstract class Shape { // abstract class
    abstract void draw(); // abstract method - no body
}
interface Printable { // interface
    void print(); // implicitly public abstract
}
class A {
    static int count; // static data
    static { count = 1; } // static block
    static void show() { } // static method
    class Inner { } // non-static inner class
    static class Nested { } // static nested class
}
```

C. Access Modifier Visibility - Master Table

Modifier	Same Class	Same Package	Subclass (Other Package)	Other Package (Non-subclass)
public	Yes	Yes	Yes	Yes
protected	Yes	Yes	Yes	No
default	Yes	Yes	No	No
private	Yes	No	No	No

D. Abstract Class vs Interface - Final Recap

Point	Abstract Class	Interface
Instantiable	No	No
Method bodies	Allowed (mix of abstract + concrete)	Only default/static/private (Java 8+); rest abstract
Fields	Any kind	public static final only
Constructor	Yes	No
Inheritance keyword	extends (one class only)	implements (multiple allowed)

E. Important Rules Not to Forget

- Illegal modifier combinations: **abstract+final**, **abstract+static**, **abstract+private** (on methods).
- Top-level class can only be **public** or **default** - never private/protected.
- Local variables take **no access modifier** and have **no default value** - must be initialized before use.
- Static methods cannot use `this`/`super` and cannot directly access instance members.
- Multiple static blocks execute in the **order they appear** in the source file, before main().
- Non-static inner class object: `Outer.Inner in = outerObj.new Inner();`
- Static nested class object: `Outer.Nested n = new Outer.Nested();` (no outer object needed).
- A class must be `public` to be usable/importable from another package.
- `import pkg.*;` does NOT import sub-packages.
- `java.lang` package is auto-imported into every Java program.
- Wrapper classes are **immutable**; compare their values with `.equals()`, not `==`.
- User-defined package compile command: `javac -d . ClassName.java`

F. Likely Exam-Style Questions (Self-Check)

- Which access modifier allows visibility within the same package only? → **default**
- Which keyword lets a class implement multiple types in Java? → **implements (interfaces)**
- When does a static block execute? → **Once, at class loading, before main()**
- Can an abstract class have a constructor? → **Yes**
- What is the default value of interface variables' modifiers? → **public static final**
- Which command compiles a class into its package folder? → **javac -d . FileName.java**
- What does autoboxing mean? → **Automatic conversion from primitive to wrapper object**

SUMMARY

Revise the tables in this section a day before the exam - they cover the highest-yield facts across all 11 syllabus topics on Language Features.